QA.GURU · PYTHON · MOBILE 2

Мобильная автоматизация

От одного облачного теста — к управляемому запуску на эмуляторе, симуляторе и настоящем устройстве.

pytest · Appium · Allure · Android · iOS

РЕЗУЛЬТАТ ЗАНЯТИЯ

Один сценарий. Разные места запуска.

01

Управлять сессией

Создать драйвер из конфигурации и завершить его даже при падении.

02

Видеть причину

Screenshot, XML, доступные логи и видео в Allure.

03

Выбирать среду

Android локально и на ферме. iOS на Mac и iPhone.

ТОЧКА СТАРТА

Продолжаем python-appium, не пишем новый фреймворк

Уже есть в локальной ветке

- pytest + Appium-Python-Client.
- Фабрика `create_driver(config)`.
- BrowserStack / Selenoid / local Android.
- Allure screenshot, XML и хук падения.

Ветка `selenoid` · состояние кода на 14.10.2026.

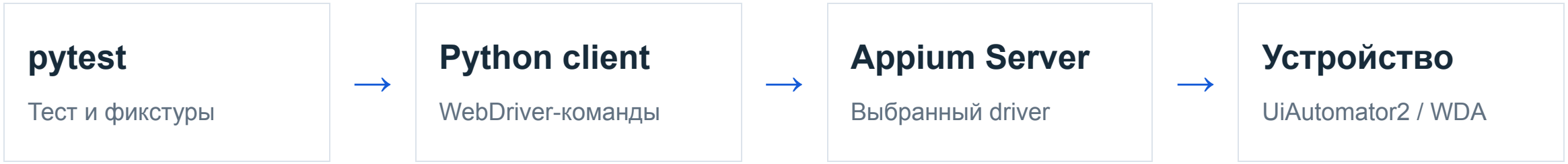
Развиваем на занятии

- Явный выбор устройства по UDID.
- Понятные профили запуска и состояния.
- Видео фермы и диагностика аттачей.
- Отдельные возможности XCUITest / iOS.

Без Selene, Sonar, API-клиента и лишних абстракций.

АРХИТЕКТУРА

Кто кому отправляет команды



Android: Appium → UiAutomator2 → приложение.

iOS: Appium → XCUITest → WebDriverAgent → приложение.

Inspector — ещё один клиент. Он подключается к серверу, а не заменяет его.

СРЕДЫ ВЫПОЛНЕНИЯ

Эмулятор Android ≠ симулятор iOS

ПРОФИЛЬ	ANDROID	IOS
emulator	Android Emulator / AVD	—
simulator	—	Xcode Simulator
real	Android + adb + UDID	iPhone + Xcode + WDA
selenoid	Учебная Android-ферма	Не поддерживается
browserstack	Существующий облачный маршрут	Отдельный контракт и IPA

Матрица — цель урока. Наличие профиля в конфигурации не доказывает успешный запуск.

Wikipedia: сохраняем преемственность

Android

- Официальный stable APK: **50611-r-2026-10-07**.
- Для stable проверяем `org.wikipedia`.
- GitHub `latest` — alpha, другой канал и package.
- Версию и SHA-256 фиксируем до урока.

Официальный APK · stable-каталог

iOS

- Simulator: сборка `.app` из исходников.
- iPhone: установленная версия из App Store.
- Для iPhone можно запускать по `bundleId`.
- Пересборка Wikipedia не обязательна для этого пути.

Исходники iOS · App Store

Фиксируем toolchain, а не устанавливаем «всё latest»

Рантаймы учебного окружения

- **Python 3.14** + venv.
- **JDK 21** для Android tooling.
- **Node.js 26**, npm 10+ для Appium.

Pins из SSOT рабочего пространства. Зависимости Python берём из учебного проекта.

Проверенные npm-версии

Appium Server	3.5.0
UiAutomator2	6.0.0
XCUITest	12.8.0
Allure CLI 2	2.34.1

Существование, возраст релизов и Appium peerDependencies проверены. Device smoke — отдельный шаг.

Изолированное окружение проекта

macOS / Linux

```
python3.14 -m venv .venv
.venv/bin/python -m pip install \
    -r requirements.txt
.venv/bin/python --version
.venv/bin/pytest --collect-only tests
```

Collection проверяет импорт и обнаружение тестов, не соединение с устройством.

Windows · PowerShell

```
py -3.14 -m venv .venv
.\.venv\Scripts\python.exe -m pip install `
    -r requirements.txt
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\pytest.exe `
    --collect-only tests
```

Можно использовать venv без Activate.ps1 и без ослабления Execution Policy.

Android Studio — удобный вход в SDK

SDK Manager

- Android SDK Platform.
- Android SDK Platform-Tools.
- Android SDK Build-Tools.
- Android SDK Command-line Tools.
- Android Emulator + system image.

Device Manager

- Создать AVD с выбранной версией Android.
- Apple Silicon: образ подходящей arm64-архитектуры.
- Intel/AMD: подходящий x86_64-образ.
- Проверить аппаратную виртуализацию.
- Дождаться полного старта Android.

JAVA_HOME, ANDROID_HOME и PATH

```
export JAVA_HOME=$(/usr/libexec/java_home -v 21)
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$JAVA_HOME/bin:$PATH"
export PATH="$ANDROID_HOME/platform-tools:$PATH"
export PATH="$ANDROID_HOME/emulator:$PATH"
export PATH="$ANDROID_HOME/cmdline-tools/latest/bin:$PATH"

java -version
adb version
emulator -list-avds
```

ПРИМЕР ДЛЯ macOS

Путь к SDK сверяем в Android Studio, а не угадываем.

Linux: укажи свой JDK и SDK; часто SDK находится в `$HOME/Android/Sdk`.

> Проверка важнее наличия переменной: команды должны реально находить нужные инструменты.

Те же инструменты, другие пути

```
$env:JAVA_HOME = "C:\Tools\jdk-21"
$env:ANDROID_HOME = "$env:LOCALAPPDATA\Android\Sdk"
$env:Path = "$env:JAVA_HOME\bin;" +
    "$env:ANDROID_HOME\platform-tools;" +
    "$env:ANDROID_HOME\emulator;" + $env:Path

java -version
adb version
adb devices -l
emulator -list-avds
```

JDK устанавливаем отдельно или используем проверенный доступный JDK 21.

Путь `C:\Tools\jdk-21` — пример. Укажи фактический каталог.

После изменения постоянных переменных открой новый терминал. Не отключай защиту системы ради настройки.

Server и drivers устанавливаются отдельно

```
npm install -g appium@3.5.0
appium driver install uiautomator2@6.0.0

appium --version
appium driver list --installed
appium driver doctor uiautomator2

appium driver install xcuitest@12.8.0
appium driver doctor xcuitest
```

UiAutomator2 — Android.

XCUITest — локальный iOS на Mac.

UiAutomator2 5+ и XCUITest 10+ требуют Appium 3.

Установки — только в выделенном учебном окружении. Версию фермы этим не меняем.

Inspector соединяется с уже работающим сервером

Appium Server

```
appium --address 127.0.0.1 \
    --base-path /wd/hub
```

Пример для существующего профиля `/wd/hub`. У Appium путь по умолчанию — `/`.

Host/port берём из выбранного учебного профиля и вывода сервера.

Appium Inspector

- Отдельное desktop-приложение.
- Скачать официальный release под свою ОС/архитектуру.
- Указать тот же host, port, SSL и base path.
- Передать capabilities выбранного устройства.
- После исследования завершить сессию.

Сначала устройство online, потом pytest

AVD

Запустить в Device Manager

→

adb

Проверить устройство

```
adb devices -l
adb -s "$ANDROID_UDID" shell getprop \
    ro.product.cpu.abi
adb -s "$ANDROID_UDID" shell getprop \
    sys.boot_completed
```

- В списке должен быть статус `device`, не offline.
- Зафиксировать serial в конфигурации.
- Проверить совместимость APK и образа.
- Запустить Appium и первый smoke.

Один эмулятор, один последовательный запуск — хороший старт для урока.

USB debugging + доверие + конкретный serial

- Включить Developer Options и USB debugging.
- Подключить телефон по USB.
- Подтвердить доверие компьютеру на телефоне.
- Убедиться: `adb devices -l` → `device`.

```
options.udid = device_udid
options.automation_name = "UiAutomator2"
options.app = apk_path
```

Не выбираем «первый попавшийся». Эмулятор и телефон могут быть подключены одновременно.

Используй учебное устройство. Очистка приложения на личном телефоне — только с согласия владельца.

Android: минимальные типизированные options

```
from appium import webdriver
from appium.options.android import UiAutomator2Options

options = UiAutomator2Options()
options.platform_name = "Android"
options.automation_name = "UiAutomator2"
options.udid = device_udid
options.app = apk_path
options.no_reset = False

driver = webdriver.Remote(
    appium_url, options=options
)
```

`appium_url` — выбранный учебный сервер.

`device_udid` — конкретный serial.

`apk_path` — файл на машине Appium Server.

> Если задаём package/activity вручную, они должны соответствовать именно этому APK.

Меняем конфигурацию, а не тест

```
config.py
    env → выбранный профиль

drivers.py
    config → Appium session

conftest.py
    fixture → setup / teardown

helpers/attach.py
    session → Allure artifacts

tests/test_search.py
    сценарий → содержательная проверка
```

Целевой контракт

- `PLATFORM`: android / ios.
- `DEVICE_HOST`: место запуска.
- Локально: файл + UDID.
- Ферма: доступный URL приложения.
- Секреты: env, не Git и не отчёт.

PLATFORM / iOS / UDID — расширение текущего проекта. Не меняем всё сразу.

СТАБИЛЬНЫЙ ТЕСТ

Ждём состояние, проверяем смысл

```
wait = WebDriverWait(driver, config.timeout)
field = wait.until(
    EC.visibility_of_element_located(
        (AppiumBy.ID,
         "org.wikipedia:id/search_src_text")
    )
)
field.send_keys("Python")
items = wait.until(
    EC.presence_of_all_elements_located(
        (AppiumBy.ID,
         "org.wikipedia:id/page_list_item_title")
    )
)
assert any("Python" in item.text for item in items)
```

- Explicit waits вместо UI `time.sleep`.
- Нулевая implicit wait.
- Локаторы сверяем в Inspector.
- Фиксируем локаль и состояние onboarding.
- «Есть любые TextView» — слабый oracle.

На iOS локаторы другие; смысл сценария сохраняется.

Упал тест. Что мы хотим увидеть?

Не «element not found»,
а экран, состояние и историю действий.

Screenshot · XML page source · шаги · доступные логи · видео

Артефакты помогают объяснить сбой, но не превращают красный тест в зелёный.

Плагин pytest пишет результаты, CLI строит отчёт

```python
@allure.title("Wikipedia: поиск Python")
def test_search(driver):
    with allure.step("Открыть поиск"):
        open_search(driver)
    with allure.step("Найти Python"):
        search(driver, "Python")
    with allure.step("Проверить результаты"):
        assert_python_found(driver)
```

Пример структуры сценария, не готовые функции проекта.

```bash
npm install -g allure-commandline@2.34.1
allure --version

.venv/bin/pytest tests \
    --alluredir=allure-results

allure generate allure-results \
    -o allure-report
```

Allure CLI 2 использует Java; JDK 21 уже нужен Android tooling.

Screenshot и page source снимаем до quit()

```
allure.attach(
    driver.get_screenshot_as_png(),
    name="Screenshot",
    attachment_type=allure.attachment_type.PNG,
)

allure.attach(
    driver.page_source,
    name="Page source",
    attachment_type=allure.attachment_type.XML,
)
```

Два разных вопроса

Screenshot: что было видно?

Page source: какие элементы и атрибуты были доступны?

XML не заменяет картинку. Картинка не заменяет дерево элементов.

Сессия завершается при любом исходе теста

```
@pytest.fixture
def driver(config):
    drv = create_driver(config)
    session_id = drv.session_id
    try:
        yield drv
    finally:
        try:
            attach.screenshot(drv)
            attach.page_source(drv)
        finally:
            drv.quit()
        attach.video(config, session_id)
```

- Запоминаем session_id заранее.
- Live-аттачи — пока драйвер жив.
- `quit()` — гарантированный cleanup.
- Remote video — после quit.

Контракт helpers: ошибки аттачей диагностируются, но не маскируют исходный сбой теста.

Screenshot в момент падения, не после reset

```
@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when != "call" or not report.failed:
        return
    drv = item.funcargs.get("driver")
    if drv is not None:
        attach.screenshot(
            drv, "screenshot on failure"
        )
```

Hook выполняется после тестового вызова, до teardown фикстуры.

Для падения setup нужна отдельная точка сбора в setup-ветке фикстуры.

Проверяем контролируемым падением: исходная ошибка сохранена, аттачи есть, сессия закрыта.

Один интерфейс, разные источники артефактов

ИСТОЧНИК	ЧТО СОБИРАЕМ	КОГДА
Локальный Android	PNG, XML, доступный logcat; запись при поддержке	Пока сессия жива
Локальный iOS	PNG, XML, доступные логи; запись при поддержке	Пока сессия жива
Selenoid	PNG/XML через driver; MP4 от фермы	Видео после quit
BrowserStack	PNG/XML; артефакты App Automate API	Remote artifacts по session_id

Нет нужного log type или записи? Сообщаем ограничение. Не называем отсутствующий файл успешно собранным.

Учебная ферма selenoid.qa.guru — только Android

```
options.set_capability("browserName", "android")
options.set_capability(
    "browserVersion", catalog_android_version
)
options.app = apk_url
options.set_capability(
    "selenoid:options",
    {"enableVNC": True, "enableVideo": True},
)

driver = webdriver.Remote(
    approved_hub_url, options=options
)
```

- Версия Android — из актуального каталога.
- APK URL должен быть доступен ферме.
- Авторизация — env, не слайды и не Git.
- Отдельные options, не весь local-профиль.

В текущем маршруте `appPackage` не совмещаем с routing `browserName=android`.

ПРАКТИКА · REMOTE VIDEO

Видео появляется не в момент screenshot

enableVideo
До создания сессии

→

session_id
Сохраняем до quit

→

quit
Ферма завершает запись

→

MP4
Получаем с timeout

```
video_path = f"/video/{session_id}.mp4"
allure.attach(
    video_bytes,
    name="Video",
    attachment_type=allure.attachment_type.MP4,
)
```

Путь — для стандартного videoName. Авторизацию и способ получения проверяем по контракту фермы.

Существующий облачный маршрут сохраняем

Конфигурация

```
BROWSERSTACK_USERNAME
BROWSERSTACK_ACCESS_KEY
BROWSERSTACK_APP

DEVICE_HOST=browserstack
```

Имена параметров из проекта. Значения секретов не показываем.

- Appium-сессия и артефакты связаны через session_id.
- Видео получаем через App Automate API.
- `bs://...` — загруженная сборка приложения.

App Live ≠ App Automate. Установка из App Store на своём iPhone не заменяет загрузку IPA в облако.

Симулятор и iPhone требуют разных сборок

Подписать WebDriverAgent
не значит пересобрать Wikipedia.

Simulator — подготовленная .app.
iPhone — App Store приложение + WDA.

Mac, полный Xcode, runtime и XCUITest

- Установить полный Xcode.
- Проверить selected developer directory.
- Установить нужный Simulator runtime.
- Проверить Xcode ↔ macOS ↔ iOS ↔ driver.
- Appium Server и XCUITest на Mac.

Command Line Tools без полного Xcode недостаточно для всей цепочки.

```
xcode-select -p
xcodebuild -version
xcrun simctl list devices available

appium driver list --installed
appium driver doctor xcuitest
```

Несовместимая матрица — причина остановиться и выбрать поддерживаемую комбинацию, не поставить всё latest.

.app для Simulator ≠ .ipa для iPhone

Simulator

- Сборка для `iphonesimulator`.
- `.app` или архив с .app.
- Подходящая архитектура host.
- Не нужен платный Developer account.

Настоящий iPhone

- Device build с совместимой подписью.
- Или установленное App Store приложение.
- UDID конкретного телефона.
- WDA требует signing/provisioning.

Сборку готовим до занятия, не всем классом live

```
xcodebuild \
    -project Wikipedia.xcodeproj \
    -scheme Wikipedia \
    -configuration Debug \
    -sdk iphonesimulator \
    -destination 'generic/platform=iOS Simulator' \
    -derivedDataPath ./build \
    CODE_SIGNING_ALLOWED=NO
```

ШАБЛОН СБОРКИ

- Выделенная копия официальных исходников.
- Зафиксированный commit/release.
- Проверенные scheme и зависимости.
- Готовая .app с проверенной архитектурой.

Upstream setup ставит линтеры и hook. Не запускаем его вслепую и не удаляем проверки.

Simulator сессия через XCUITestOptions

```
from appium import webdriver
from appium.options.ios import XCUITestOptions

options = XCUITestOptions()
options.platform_name = "iOS"
options.automation_name = "XCUITest"
options.udid = simulator_udid
options.app = simulator_app_path
options.no_reset = False

driver = webdriver.Remote(
    appium_url, options=options
)
```

`simulator_udid` — из simctl/Xcode.

`simulator_app_path` — проверенная .app на Mac-сервере.

> Android `resource-id` нельзя перенести на iOS. Используй Inspector и accessibility identifiers.

Учебное расширение: iOS factory в текущем проекте ещё нет.

Сначала доверие и WDA, потом тест Wikipedia

1. Подключить iPhone и выполнить pairing в Xcode.
2. На телефоне подтвердить Trust This Computer.
3. Включить Developer Mode.
4. Проверить UI Automation в Developer settings.
5. Выбрать Apple Account / Team для WDA.
6. Подписать и проверить первый запуск WDA.

```
appium driver run xcuitest open-wda
```

Personal Team: ручная настройка, ограничения и срок действия provisioning.

Paid account: доступна автоматическая конфигурация согласно документации.

Доверие, аккаунт и подпись подтверждает владелец. Никакого обхода 2FA и trust checks.

На iPhone не обязательно собирать собственную IPA

```
options = XCUITestOptions()
options.platform_name = "iOS"
options.automation_name = "XCUITest"
options.udid = iphone_udid
options.bundle_id = installed_bundle_id
options.no_reset = True

driver = webdriver.Remote(
    appium_url, options=options
)
```

- Установить Wikipedia из App Store.
- Проверить фактический bundleId.
- Использовать предварительно настроенный WDA.
- Управлять состоянием через UI.

`app` не передаём: приложение уже установлено. Не делаем fullReset с последующей «магической» переустановкой.

ДИАГНОСТИКА

Ищем сбой на нужном участке цепочки

СИМПТОМ	ПРОВЕРЯЕМ
Не создаётся сессия	Endpoint/base path, driver, platform, capabilities, доступ
adb: unauthorized / offline	Доверие на телефоне, подключение, полный boot
Приложение не установилось	Файл на сервере, URL с фермы, ABI, SDK, подпись
Element not found	PNG + XML, onboarding, локаль, версия приложения
xcodebuild: 65 / 70	WDA signing/provisioning, Xcode/OS configuration
Нет remote video	enableVideo, session_id, quit, публикация и авторизация

СЦЕНАРИЙ ДЕМОНСТРАЦИИ

Один тест переносим по маршрутам

Основная практика

1. Открыть текущий тест и config/factory.
2. Inspector: проверить экран и локатор.
3. Запустить поиск на Android Emulator.
4. Показать содержательный assert.
5. Контролируемое падение → Allure.
6. Тот же сценарий → Android Selenoid.

Демонстрации преподавателя

- Android Emulator → настоящий Android по UDID.
- Simulator с подготовленной Wikipedia .app.
- iPhone с App Store Wikipedia и WDA.
- Сравнить аттачи и возможности backend.

Меняем host/platform и локаторы, не смысл сценария. Студентам без Mac доступна Android-практика.

ДОМАШНЕЕ ЗАДАНИЕ

Небольшой проект, понятные доказательства

Обязательная часть · Android

- 2–4 содержательных теста Wikipedia.
- Emulator и один разрешённый удалённый профиль.
- Конфигурация без секретов в Git.
- Allure: шаги, screenshot, XML.
- Видео там, где оно поддерживается.
- Инструкция запуска с закреплённой APK.

Дополнительная часть

- Явный UDID настоящего Android.
- iOS Simulator с подготовленной .app.
- iPhone/App Store + настроенный WDA.
- Короткое объяснение различий среды и аттачей.

Нет устройства или доступа? Укажи BLOCKED и что нужно для запуска. Не подменяй E2E моками.

ПЕРЕД ЗАНЯТИЕМ

Checklist преподавателя

Приложение и устройства

- APK: версия, SHA-256, ABI, локаторы.
- AVD запущен, adb видит нужный serial.
- Учебная ферма доступна и разрешена.
- Simulator .app реально запускается.
- WDA настроен для конкретного iPhone.

Код и отчёт

- Подтверждён стартовый commit и scope.
- Один smoke на каждом показываемом маршруте.
- Красный тест сохраняет исходную ошибку.
- PNG/XML/MP4 открываются в отчёте.
- Сессии завершаются, секретов в артефактах нет.

Официальные документы, а не угадывание

- qa-guru/python-appium
- Appium 3: system requirements
- UiAutomator2: requirements и capabilities
- Appium Inspector: установка
- XCUITest: матрица совместимости
- WDA: provisioning и Apple Account
- XCUITest: pre-installed app и Simulator
- Wikipedia iOS: исходники и сборка
- BrowserStack: контракт приложения
- WDIO demo: APK + Simulator .app

WDIO demo — запасной вариант для Android и iOS Simulator. Готового physical-iPhone build нет.